

Prompt for Claude Code — Flight Scouting & Booking Assistant Website

Build a full-stack web app for a travel booking side-business (starting small, but architected to scale into a full travel agency later). It helps me search, compare, and sort flights from **any origin to any destination worldwide** — not just Ghana — so I can quickly find good options for clients and send them booking links.

Tech stack (required)

- **Frontend:** HTML, CSS, vanilla JavaScript (no framework required — keep it simple and readable, but structure it cleanly into components/files so it's easy to extend later)
- **Backend:** Python with FastAPI
- **Deployment:** Railway (set up `railway.json` / `Procfile` as needed, and give me clear deploy instructions for both frontend and backend)
- Use environment variables for all API keys/secrets — never hardcode them

Core features

1. **Flight search form:** origin, destination (any IATA airport/city worldwide, with autocomplete/search-as-you-type), departure date, return date or one-way, passenger count, cabin class.
2. **Flexible dates mode:** show a price grid across a date range (e.g., $\pm 3-7$ days) to spot the cheapest days to fly.
3. **Results view:** airline, price, total duration, number of stops, layover airports/times, departure/arrival times, baggage allowance (where the API provides it).
4. **Sort & filter:** by price, duration, stops, airline, or price range.
5. **Shortlist/save tool:** bookmark flight options with a client name/note attached, so I can group options per client and revisit them.
6. **Currency toggle:** at minimum GHS and USD, ideally a few more (EUR, TRY, CNY) since destinations vary.
7. **Deep links:** send me straight to the airline's or OTA's booking page for the selected

flight — no need to build in-app checkout/payments.

8. **Mobile-responsive:** I'll be checking this from my phone often.
9. **Admin-friendly structure:** since this may grow into a multi-user agency tool later, keep the backend organized (separate routers/services for search, shortlist, clients) so I can add user accounts, multiple agents, and a client database down the line without a rewrite.

Data sources — use legitimate, ToS-compliant options

Don't scrape airline or OTA websites directly (breaks most airlines' and Google Flights' terms of service). Instead, integrate with real flight-search APIs. Set the app up so it can pull from **multiple providers** for broader coverage, with placeholder API keys and clear setup instructions for each:

- **Duffel** — modern API, good airline coverage, has a free sandbox/test mode, self-serve signup.
- **Amadeus for Developers (Self-Service APIs)** — very broad global airline/GDS coverage, free tier for testing, more complex data format.
- **Kiwi.com (Tequila API)** — aggregates budget/low-cost carriers well, good for wider route coverage; requires approval for production use.
- **Skyscanner Partner API / Travelpayouts** — useful as an affiliate-style option with deep links to book, and can also monetize via commission later (worth flagging as a future revenue stream for the agency idea).

Structure the backend so switching or combining providers is easy (e.g., an abstract `FlightProvider` interface with one implementation per API), rather than hardwiring the whole app to one vendor.

Nice-to-haves (only if time allows)

- Price-drop alerts (email or WhatsApp notification)
- Multi-city itinerary search
- A printable/shareable summary of a flight option formatted for sending via WhatsApp to clients
- Simple client database (name, phone, past searches/bookings)

Deliverables

- Working local dev setup instructions (README)
- Railway deployment instructions for both frontend and backend
- `.env.example` listing all required API keys
- Clean, commented code since a non-specialist may need to tweak it later